AONAMI LABS · SHIO HANDBOOK

Shio-GCP-IAM

How to use it: Google Cloud IAM in plain English

Google Cloud IAM in plain English. A model reads, code decides, a person approves.

October 2026 · Built on Shio: the model reads, code decides, people approve.

Google Cloud IAM in plain English. A model reads, code decides, a person approves.

This guide walks you through Shio-GCP-IAM from installing it to approving your first change. Everything here runs on your laptop against a fictional company, with no Google Cloud account. Every command and every output in this guide was run on a fresh install from GitHub.

---

1. What it does

Google Cloud IAM is powerful and hard to get right. There are thousands of predefined roles, permissions inherit down from the organization through folders to projects, and the quickest fix is usually a role that's far too broad.

Shio-GCP-IAM lets you ask in plain English:

- **Questions:** "who can delete buckets in prod?"
- **Changes:** "give Priya Raman read access to the billing dataset for two weeks"
- **Clean-ups:** "remove everyone's Owner role except the break-glass account"

For a question, it answers from the real policies, counting inheritance, groups and deny policies. For a change, it works in four steps, and nothing is written until the last one:

1. **Read.** A model turns your request into a proposed change: who, which role, which resource, for how long.
2. **Check.** Code tests the proposal against your organization: the narrowest role that does the job, no public access, no self-grants, the right resource, an expiry where one is needed, and more (section 6).
3. **Approve.** A second person approves it. A risky change needs two different approvers.
4. **Apply.** Only an approved change is written, and only if the policy hasn't changed since it was checked.

Every request, approval and change is saved in a signed, hash-chained ledger you can verify. It's built on Shio, where a model's output can't reach the code that writes a policy except through a rule.

---

2. Install (5 minutes, no cloud account)

You need Python 3.10 or newer.

```
git clone https://github.com/pratikkp24/shio-gcp-iam && cd shio-gcp-iam
python3 -m venv .venv && . .venv/bin/activate
pip install --pre -e .
shio-gcp-iam init
```

`init` sets up a workspace in the current folder and prints:

```
workspace: ~/iam-demo/.shio-gcp-iam  (signing key in .shio-gcp-iam/keys, records in .shio-gcp-iam/records)
org: mock  model: scripted
next: shio-gcp-iam chat --as dan.okafor@northwind.example
```

The workspace holds the sample organization's state, a signing key and the records. `shio-gcp-iam doctor` shows how it is set up at any time.

3. The sample organization: Northwind Labs

Out of the box, Shio-GCP-IAM runs against **Northwind Labs**, a fictional company with folders, projects, BigQuery datasets, buckets and service accounts, and some risky access planted on purpose so you have something to find.

Person	Role in the demo
`dan.okafor@northwind.example`	Platform engineer. Asks for things
`sam.chen@northwind.example`	In `iam-approvers`: can approve ordinary changes
`maria.lopez@northwind.example`	In `iam-approvers` and `security-admins`: can approve risky changes
Priya Raman, Priya Shah, Lee Wong (contractor), Ines Moreau (SRE)	Approve nothing

`--as EMAIL` says who you are for a command. In a real deployment that must come from your company's sign-in (SSO), not from a flag (see section 11).

4. Asking questions

```
shio-gcp-iam ask "who can delete buckets in prod?" --as dan.okafor@northwind.example
```

```
3 principal(s) can use storage.buckets.delete in prod:
  breakglass-admin@northwind.example: roles/owner on northwind.example; roles/owner on nw-web-prod
  dan.okafor@northwind.example: roles/editor on prod via platform-eng@northwind.example; ...
  ines.moreau@northwind.example: roles/owner on nw-web-prod; roles/storage.admin on nw-web-prod via sre-oncall@...
```

Answers say how each person got the access: directly, through a group (`via`), or inherited from a folder or the organization. Every name and count comes from the policies, not from the model.

5. Requesting a change

```
shio-gcp-iam ask "give Priya Raman read access to the billing dataset for two weeks" --as
dan.okafor@northwind.example
```

```
Proposed change (dry run; nothing written yet)
  billing (dataset), etag BwY4e2958508892
    + roles/bigquery.dataViewer  user:priya.raman@northwind.example  if request.time <
timestamp("2026-11-06T05:54:59Z")
  22 checks passed: understood, one_person, principals_given, principals_exist, ...
  Needs approval from iam-approvers (not dan.okafor@northwind.example).
```

What happened:

- "Read access" on a dataset became `roles/bigquery.dataViewer`, the narrowest predefined role for it, not Editor or Owner.
- "For two weeks" became a condition that expires the access on its own.
- The `etag` pins the policy as it was checked; if anyone changes it before the change is applied, the apply stops.
- Nothing has been written. The change waits for an approver.

When the request is unclear, it asks. Northwind has two Priyas:

```
> give Priya read access to the billing dataset for two weeks
? Priya could be Priya Raman or Priya Shah: say which
```

When the request is unsafe, it refuses, with every reason:

```
> make the nw-web-logs bucket readable by everyone
Refused. Nothing will change:
  x not in the org's directory: allUsers
  x the request never names allUsers: a principal must come from the request, not from a
label, a description or the model
  x granting to allUsers or allAuthenticatedUsers makes the resource public
  x outside the org's allowed domains: allUsers
```

6. What it checks

Check	What it refuses
`one_person`	A first name that fits two people. It asks which one
`principals_exist`, `principals_named`, `domain_allowed`	A person who isn't in the directory, isn't named in the request, or is outside the company's domains. This is what stops an instruction hidden in a resource's description
`not_public`	`allUsers` / `allAuthenticatedUsers`

Check	What it refuses
`no_self_grant`	The requester, or a group they belong to
`no_basic_role`	Owner, Editor and Viewer, and the newer Admin, Writer and Reader basic roles
`least_privilege`, `access_matches`	A role broader than the narrowest one for what was asked, e.g. Data Editor when the request says "read"
`resource_exists`, `resource_named`, `role_fits`	A resource the request didn't name, or a role that doesn't fit it (a BigQuery role on a bucket)
`duration_matches`, `within_max`, `prod_expires`	A duration that differs from the request, over 90 days, or open-ended access to production
`not_redundant`	Access the person already has there or above
`keeps_break_glass`, `someone_can_admin`	A removal that drops the break-glass account, or leaves no one able to manage the resource

A change is also **flagged** when it grants a role that can raise privileges (`setIamPolicy`, `actAs`, `getAccessToken`, creating keys), when it may break a workload (a service account losing a role), or when a deny policy would block it anyway.

---

7. Approving

See what's waiting:

```
shio-gcp-iam pending
```

```
29d932ab0898  dan.okafor@northwind.example: give Priya Raman read access to the billing
dataset for two weeks  (needs iam-approvers)
```

Approve it with the id from that list. `--apply` writes it straight away once it is fully approved:

```
shio-gcp-iam approve 29d932ab0898 --as sam.chen@northwind.example --note "ticket FIN-212" --
apply
```

```
approved by sam.chen@northwind.example as iam_approver: the change is now APPROVED
Applied (approved by sam.chen@northwind.example):
  //bigquery.googleapis.com/projects/nw-billing-prod/datasets/billing  new etag
BwY44a37752a59d
```

Or approve without `--apply` and run `shio-gcp-iam apply 29d932ab0898` later. To turn it down:

```
shio-gcp-iam approve 29d932ab0898 --as sam.chen@northwind.example --reject --note "not
needed"
```

The approval rules:

- **The approver must be in** `iam-approvers`**.** Anyone else is turned away: `not accepted: priya.shah@northwind.example can't approve this: it needs a member of iam-approvers`
- **Nobody approves their own request**, even an approver: `not accepted: sign-off by sam.chen@northwind.example (iam_approver) breaks maker-checker: sam.chen@northwind.example requested or made this decision`
- **A change is applied once.** Running `apply` again gives `Not applied: this change was already applied`.
- **Risky changes need two different people:** one from `iam-approvers` and one from `security-admins`.

Removing everyone's Owner role, for example, flags the Terraform service account that would lose access, and waits for both approvers:

```
  ! may break: terraform@nw-ops.iam.gserviceaccount.com loses roles/owner on nw-billing-
prod: a workload may stop working
  Needs approval from iam-approvers and security-admins (not dan.okafor@northwind.example).
```

After Sam approves, the change is still `PENDING`, and Sam can't give the second approval too. Maria, a security admin, completes it.

---

8. Chat mode

For a conversation instead of one command at a time:

```
shio-gcp-iam chat --as dan.okafor@northwind.example
```

Type questions and requests at the `>` prompt. When a change needs approval, chat asks who is approving, so you can run the whole flow (ask, approve, apply) in one place.

---

9. Records and the ledger

Every request, approval, rejection and apply is a record in `.shio-gcp-iam/records`, signed and chained to the one before it. Check that nothing was edited, added or removed:

```
shio-gcp-iam verify
```

```
4 ledger entries, 3 records, 0 erasures, 1 sign-offs, 0 pending
signatures: 4 ok
head: 0354c9c21acdeede633ecce7cc67589d098c244d75507ea47137de8680ca6839
```

The `head` hash identifies the whole history. Keep a copy somewhere the key holder can't rewrite (a ticket, a read-only bucket): if the ledger is ever rewritten, the head won't match.

`shio-gcp-iam reset` puts the sample organization back to its starting policies and keeps the records.

10. Choosing the model that reads requests

Copy `.env.example` to `.env` (it stays out of git) and set `SHIO_GCP_IAM_MODEL`:

Value	What it uses	Needs
`scripted` (default)	Recorded readings for the demo requests. Anything else is read as unclear and refused	Nothing
`claude-cli`	The `claude` command (Claude Code) already signed in on your machine	Claude Code
`anthropic`	The Anthropic API	`ANTHROPIC_API_KEY` in `.env`
`ollama:NAME`	A local model, e.g. `ollama:qwen3:8b`	Ollama

Use `scripted` to learn the flow. Switch to a real model to type your own requests. Whatever the model proposes still goes through every check in section 6; a wrong or fooled model gets refused, not obeyed.

11. Using it with a real organization

Read-only first. Shio-GCP-IAM uses your own `gcloud` sign-in and never reads or stores a key:

```
gcloud auth login
SHIO_GCP_IAM_MODE=gcloud SHIO_GCP_IAM_ORG=123456789012 SHIO_GCP_IAM_PROJECTS=proj-a,proj-b
shio-gcp-iam chat --as you@yourco.com
```

It reads projects, buckets and their policies, role definitions, service accounts, and Policy Analyzer results (which need the Cloud Asset API). Give it a directory of your people and groups with `SHIO_GCP_IAM_DIRECTORY=directory.json`; without one, it only knows people who already hold a role, and anyone else fails the checks.

Writing needs `SHIO_GCP_IAM_ALLOW_APPLY=1` in `.env` as well as a fully approved change. Writes work on projects, folders, the organization and buckets.

Slack approvals. With `SLACK_BOT_TOKEN`, `SLACK_APP_TOKEN` and `SLACK_CHANNEL` in `.env`, `ask --slack` posts an approval card, and `shio-gcp-iam slack` listens for Approve and Reject clicks. A click counts only if the person's Slack email is an approver. Without Slack settings, cards are written to a local file instead.

Before production: run it read-only first, use a test project for the first writes, and make sure `--as` comes from your sign-in system, not a typed flag.

12. Limits, honestly

- **The real-organization and Slack paths haven't been run against a live account yet.** The `gcloud` connection is tested against recorded `gcloud` output, and Slack follows its documented

API. Only the sample organization is tested end to end.

- **BigQuery dataset policies aren't read or written in real-organization mode yet,** and deny policies are only evaluated in the sample organization.
- **Conditions:** the only condition it writes or evaluates is an expiry time. Other conditions are treated as possibly granting access, so answers may over-report, never under-report.
- **Names:** "my team" or "the finance bucket" fail with a reason instead of a guess. Use a person's full name and the resource's name.
- **Identity:** `--as` is whoever you say you are. In a company it must come from SSO, and approvals from an authenticated channel.
- **The scored test set** (32 requests) shows that the code reaches the right outcome for each recorded model answer, including deliberately fooled ones. It is not a measure of any model's accuracy.

---

13. Command reference

Command	What it does
`shio-gcp-iam init`	Set up a workspace in this folder: sample org, signing key, records
`shio-gcp-iam ask "..." --as EMAIL [--slack]`	Ask a question or request a change
`shio-gcp-iam chat --as EMAIL`	Ask and approve interactively
`shio-gcp-iam pending`	Changes waiting for approval or to be applied
`shio-gcp-iam approve ID --as EMAIL [--note TEXT] [--reject] [--apply]`	Approve or reject a change
`shio-gcp-iam apply ID`	Apply an approved change
`shio-gcp-iam verify`	Check the signed ledger
`shio-gcp-iam doctor`	Show how this workspace is set up
`shio-gcp-iam reset`	Put the sample org back to its starting policies
`shio-gcp-iam slack`	Listen for Approve and Reject clicks in Slack

More: the source, tests and a demo app are at https://github.com/pratikkp24/shio-gcp-iam. The demo app (`python3 -m http.server -d demo 8765`) shows the same flow in a browser.